

Freestanding Library: Rewording the Status Quo

Document number: P1641R2

Date: 2020-01-10

Reply-to: Ben Craig <ben dot craig at gmail dot com>

Audience: SG14, Library Evolution Working Group

Change history

R2

- Rebased to N4842
- Accounting for addition / changes to version.syn
- Accounting for addition of <compare>
- Added wording for freestanding-only macros

R1

- Rebased to N4830
- Dealt with synchronization library additions
- Dealt with source_location addition
- Updated feature test macros
- Made new feature test macro name more consistent with other library names

R0

Branching from P0829R4. This "omnibus" paper is still the direction I am aiming for. However, it is too difficult to review. It needs to change with almost every meeting. Therefore, it is getting split up into smaller, more manageable chunks.

Limiting paper to <version>, blanket wording, and freestanding facilities already in the working draft.

Introduction

This paper is the first of many smaller papers adding facilities to the C++ freestanding library. This paper in particular will be updating the editorial technique for declaring facilities as freestanding and adjusting feature test macros. This update will make it easier to mark headers as partially freestanding. This wording change will make it easier to mark in-flight proposals as freestanding, without causing a major blocking point in the standardization process.

Motivation and Design

Many existing facilities in the C++ standard library could be used without trouble in freestanding environments. This series of papers will specify the maximal subset of the C++ standard library that does not require an OS or space overhead.

For a more in depth rationale, see P0829.

This paper is going to add blanket wording to make it easier to mark things freestanding, and it will retrofit the existing freestanding facilities. It also includes a non-normative note that should allow implementers to use existing no-exception builds of their libraries as an implementation defined extensions, with a fair number of qualifications. Here's that note, reproduced so that it can be debated more effectively:

[*Note:* Throwing a standard library provided exception is not observably different from `terminate()` if the implementation doesn't unwind the stack during exception handling and the user's program contains no catch blocks. *-end note*]

The one intentional design change in this paper is in the feature test macros. A new feature test macro will be added, and the existing library feature test macros will be annotated as required in freestanding or not.

Feature Test Macro

A freestanding implementation that provides support for this paper shall define the following feature test macro.

Name	Value	Header
<code>__cpp_lib_freestanding</code>	<i>202001</i>	<code><version></code>

Example usage:

```
#if __cpp_lib_freestanding >= 202001
int my_atoi(const char *str);
#else
#include <cstdlib>
inline int my_atoi(const char *str) {return std::atoi(str);}
#endif
```

The `__cpp_lib_freestanding` macro is useful for detecting the absence of throwing facilities. A user could conditionally use a hand-rolled implementation in freestanding mode or the standard implementation in hosted mode.

The library feature macros have been partitioned into those that must be present in both freestanding and hosted mode, and those that only need to be present in hosted mode. If the implementation provides more than the minimal freestanding subset, then the implementation shall also provide the corresponding feature test macros.

Example of a freestanding user detecting the presence of an optional feature:

```
#if __cpp_lib_addressof_constexpr >= 201603
#include <memory>
template <class T>
constexpr std::decay_t<T>* my_addressof(T && arg) {
    return std::addressof(std::forward<T>(arg));
}
#else
template <class T>
constexpr std::decay_t<T>* my_addressof(T && arg) {
    return &arg;
}
#endif
```

Wording

The following changes are relative to N4842 from the Post-Belfast 2019 mailing.

Add a new subclause [freestanding.membership], under [conventions] and after [objects.within.classes]:

?????

Freestanding membership

[freestanding.membership]

- 1 Freestanding implementations have several declarations and macro definitions that shall meet the same requirements as for a hosted implementation unless otherwise specified.
[Note: Throwing a standard library provided exception is not observably different from terminate(.) if the implementation doesn't unwind the stack during exception handling and the user's program contains no catch blocks. -end note]
In the associated header synopsis for such declarations and macro definitions, the items shall be followed with a comment that ends with *freestanding*, as in:

#define E2BIG see below //freestanding
- 2 Freestanding implementations have several headers that shall meet the same requirements as for a hosted implementation unless otherwise specified. The synopsis for these headers shall start with a comment that ends with *freestanding*, as in:

//freestanding
namespace std {
- 3 Freestanding implementations have some macro definitions that shall not be present on hosted implementations. In the associated header synopsis for such macro definitions, the items shall be followed with a comment that ends with *freestanding only*, as in:

#define cpp lib freestanding 202001L //freestanding only
- 4 Deduction guides for *freestanding class templates* shall be implemented in freestanding implementations. Freestanding class templates are class templates that are implemented (partially or fully) in freestanding implementations.
- 5 The containing namespace of each *freestanding declaration* shall be provided in a freestanding implementation. Freestanding declarations are non-namespace declarations that are implemented (partially or fully) in a freestanding implementation

Change in [compliance] paragraph 3:

~~The supplied version of the header `<cstdlib>` shall declare at least the functions `abort`, `atexit`, `at_quick_exit`, `exit`, and `quick_exit` ([support.start.term]). The supplied version of the header `<atomic>` shall meet the same requirements as for a hosted implementation except that support for always lock-free integral atomic types ([atomics.lockfree]) is implementation-defined, and whether or not the type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` are defined ([atomics.alias]) is implementation-defined. The other headers listed in this table shall meet the same requirements as for a hosted implementation.~~ **The headers listed in this table shall meet the requirements for a freestanding implementation, as specified in the respective header synopsis.**

Instructions to the editor:

Please add a *//freestanding* comment to the beginning of the following synopses. These headers are entirely freestanding.

- **[cstddef.syn]**

- [\[limits.syn\]](#)
- [\[climits.syn\]](#)
- [\[cfloat.syn\]](#)
- [\[cstdint.syn\]](#)
- [\[new.syn\]](#)
- [\[typeinfo.syn\]](#)
- [\[source_location.syn\]](#)
- [\[exception.syn\]](#)
- [\[initializer.list.syn\]](#)
- [\[compare.syn\]](#)
- [\[coroutine.syn\]](#)
- [\[cstdarg.syn\]](#)
- [\[concepts.syn\]](#)
- [\[meta.type.synop\]](#)
- [\[bit.syn\]](#)

Change in [\[cstdlib.syn\]](#):

???	Header <code><cstdlib></code> synopsis	[cstdlib.syn]
-----	---	--------------------------------------

```

namespace std {
    using size_t = see below; // freestanding
    using div_t = see below;
    using ldiv_t = see below;
    using lldiv_t = see below;
}

#define NULL see below // freestanding
#define EXIT_FAILURE see below
#define EXIT_SUCCESS see below
#define RAND_MAX see below
#define MB_CUR_MAX see below

namespace std {
    // Exposition-only function type aliases
    extern "C" using c-atexit-handler = void(); // exposition only
    extern "C++" using atexit-handler = void(); // exposition only
    extern "C" using c-compare-pred = int(const void*, const void*); // exposition only
    extern "C++" using compare-pred = int(const void*, const void*); // exposition only

    // [support.start.term], start and termination
    [[noreturn]] void abort() noexcept; // freestanding
    int atexit(c-atexit-handler* func) noexcept; // freestanding
    int atexit(atexit-handler* func) noexcept; // freestanding
    int at_quick_exit(c-atexit-handler* func) noexcept; // freestanding
    int at_quick_exit(atexit-handler* func) noexcept; // freestanding
    [[noreturn]] void exit(int status); // freestanding
    [[noreturn]] void _Exit(int status) noexcept;
    [[noreturn]] void quick_exit(int status) noexcept; // freestanding

    char* getenv(const char* name);
    int system(const char* string);

    // [c.malloc], C library memory allocation

```

```

void* aligned_alloc(size_t alignment, size_t size);
void* calloc(size_t nmemb, size_t size);
void free(void* ptr);
void* malloc(size_t size);
void* realloc(void* ptr, size_t size);

double atof(const char* nptr);
int atoi(const char* nptr);
long int atol(const char* nptr);
long long int atoll(const char* nptr);
double strtod(const char* nptr, char** endptr);
float strtof(const char* nptr, char** endptr);
long double strtold(const char* nptr, char** endptr);
long int strtol(const char* nptr, char** endptr, int base);
long long int strtoll(const char* nptr, char** endptr, int base);
unsigned long int strtoul(const char* nptr, char** endptr, int base);
unsigned long long int strtoull(const char* nptr, char** endptr, int base);

// [c.mb.wcs], multibyte / wide string and character conversion functions
int mblen(const char* s, size_t n);
int mbtowc(wchar_t* pwc, const char* s, size_t n);
int wctomb(char* s, wchar_t wchar);
size_t mbstowcs(wchar_t* pwcs, const char* s, size_t n);
size_t wcstombs(char* s, const wchar_t* pwcs, size_t n);

// [alg.c.library], C standard library algorithms
void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
              c-compare-pred* compar);
void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
              compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, c-compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, compare-pred* compar);

// [c.math.rand], low-quality random number generation
int rand();
void srand(unsigned int seed);

// [c.math.abs], absolute values
int abs(int j);
long int abs(long int j);
long long int abs(long long int j);
float abs(float j);
double abs(double j);
long double abs(long double j);

long int labs(long int j);
long long int llabs(long long int j);

div_t div(int numer, int denom);
ldiv_t div(long int numer, long int denom); // see [library.c]
lldiv_t div(long long int numer, long long int denom); // see [library.c]
ldiv_t ldiv(long int numer, long int denom);
lldiv_t lldiv(long long int numer, long long int denom);
}

```

Change in [version.syn]:
Instructions to the editor:

Please add the following macro definition to [version.syn]

```
#define __cpp_lib_freestanding_202001L //freestanding only
```

Please add a *//freestanding* comment to the following macro definitions:

- `__cpp_lib_atomic_flag_test`
- `__cpp_lib_atomic_float`
- `__cpp_lib_atomic_is_always_lock_free`
- `__cpp_lib_atomic_ref`
- `__cpp_lib_atomic_value_initialization`
- `__cpp_lib_atomic_wait`
- `__cpp_lib_bit_cast`
- `__cpp_lib_bitops`
- `__cpp_lib_bool_constant`
- `__cpp_lib_bounded_array_traits`
- `__cpp_lib_byte`
- `__cpp_lib_char8_t`
- `__cpp_lib_concepts`
- `__cpp_lib_destroying_delete`
- `__cpp_lib_endian`
- `__cpp_lib_hardware_interference_size`
- `__cpp_lib_has_unique_object_representations`
- `__cpp_lib_int_pow2`
- `__cpp_lib_integral_constant_callable`
- `__cpp_lib_is_aggregate`
- `__cpp_lib_is_constant_evaluated`
- `__cpp_lib_is_final`
- `__cpp_lib_is_invocable`
- `__cpp_lib_is_layout_compatible`
- `__cpp_lib_is_null_pointer`
- `__cpp_lib_is_pointer_interconvertible`
- `__cpp_lib_is_swappable`
- `__cpp_lib_laundry`
- `__cpp_lib_logical_traits`
- `__cpp_lib_nothrow_convertible`
- `__cpp_lib_remove_cvref`
- `__cpp_lib_result_of_sfinae`
- `__cpp_lib_source_location`
- `__cpp_lib_three_way_comparison`
- `__cpp_lib_transformation_trait_aliases`
- `__cpp_lib_type_identity`
- `__cpp_lib_type_trait_variable_templates`
- `__cpp_lib_uncaught_exceptions`
- `__cpp_lib_void_t`

Instructions to the editor:

Please add a *//freestanding* comment to all entities in [atomics.syn] other than `atomic_signed_lock_free` and `atomic_unsigned_lock_free`

Change in [atomics.lockfree] paragraph 2:

On hosted implementations ([compliance]), at least one signed integral specialization of the atomic template, along with the specialization for the corresponding unsigned type ([basic.fundamental]), is always lock-free.

[*Note*

: This requirement is optional in freestanding implementations ([compliance]). — *end note*

Acknowledgements

Thanks to Brandon Streiff, Joshua Cannon, Phil Hindman, and Irwan Djajadi for reviewing P0829.

Thanks to Odin Holmes for providing feedback and helping publicize P0829.

Thanks to Paul Bendixen for providing feedback while prototyping P0829.

Similar work was done in the C++11 timeframe by Lawrence Crowl and Alberto Ganesh Barbati in [N3256](#).